Generazione ed Interpolazione Temporale di Frame Video ad Alta Risoluzione: Analisi Prestazionale e Modelli di Ottimizzazione Parallela in Architettura NVIDIA CUDA

Elio Scaramuzzino
Università degli Studi di Firenze
Laboratorio Parallel Computing

***Sommario*—Questo lavoro presenta la progettazione, l'implementazione e l'analisi prestazionale comparativa di un sistema di generazione ed interpolazione di fotogrammi video intermedi operante su risoluzioni 4K UHD. L'obiettivo primario risiede nell'indagine quantitativa dei benefici architetturali derivanti dalla parallelizzazione massiva mediante framework NVIDIA CUDA rispetto ad un baseline sequenziale CPU single-thread. Il nucleo computazionale è costituito dalla stima del movimento locale bidimensionale mediante macro-blocchi (Block Matching con metrica SAD) e dall'interpolazione spaziotemporale tramite spline cubiche di Catmull-Rom con backward warping e ricampionamento bicubico. L'analisi sperimentale condotta su architettura NVIDIA Pascal (GTX 1060 6GB) evidenzia speedup superiori a $122\times$ per le componenti compute-bound ottimizzate in Shared Memory, delineando con rigore il comportamento dei kernel sotto vincoli di memoria globale, tiling cooperativo e sincronizzazione warp-level.**

I. Introduzione ed Inquadramento Teorico

Nel contesto dell'elaborazione di sequenze video ad altissima risoluzione (3840×2160), l'interpolazione temporale di fotogrammi (*Video Frame Interpolation*, VFI) costituisce un paradigma ad elevato costo computazionale. Il carico algoritmico cresce quadraticamente con la finestra di ricerca del moto e linearmente con la densità di pixel, rendendo l'approccio sequenziale inadeguato per carichi interattivi o ad alto throughput.

La pipeline implementata opera su quaterne di frame consecutivi $\{I_{-1}, I_0, I_1, I_2\}$ campionati a passi temporali costanti $t \in \{-1, 0, 1, 2\}$, con l'obiettivo di sintetizzare n frame intermedi I_τ per valori continui di tempo $\tau \in (0, 1)$ uniformemente distribuiti tra I_0 e I_1.

A. Interpolazione Lineare Elementare (LERP)

Come baseline analitica per carichi memory-bound, è stato implementato l'algoritmo LERP:

$$I_\tau(x, y) = (1 - \tau) \cdot I_0(x, y) + \tau \cdot I_1(x, y) \tag{1}$$

Tale approccio richiede un numero minimo di operazioni in virgola mobile per byte trasferito, operando come test di saturazione della banda di memoria.

B. Stima del Moto: Block Matching SAD

Al fine di preservare la consistenza geometrica delle traiettorie tra I_0 e gli altri fotogrammi della sequenza, l'immagine viene decomposta in macro-blocchi quadrati di dimensione $B \times B$ pixel. Per ciascun blocco si stima il vettore di spostamento bidimensionale $d = (u, v)$ che minimizza la *Sum of Absolute Differences* (SAD) all'interno di una finestra di ricerca isotropa $[-W, W]$:

$$\mathrm{SAD}(u, v) = \sum_{i,j=0}^{B-1} \sum_{c \in \{R,G,B\}} \Big| I_0(x_0 + j, y_0 + i)_c - I_k(x_0 + u + j, y_0 + v + i)_c \Big| \tag{2}$$

con $(u, v) \in [-W, W]^2$. Tale procedura è calcolata indipendentemente per i target I_{-1}, I_1, I_2, producendo i vettori d_{-1}, d_1, d_2 (ponendo per definizione $d_0 = (0, 0)$).

C. Interpolazione Temporale mediante Spline Catmull-Rom

Lo spostamento al tempo frazionario $\tau \in [0, 1]$ non segue una cinematica lineare. Per garantire la continuità della velocità, lo spostamento continuo $D(\tau) = (D_x(\tau), D_y(\tau))$ è calcolato applicando la formulazione canonica della spline cubica di Catmull-Rom sui quattro campioni temporali $\{d_{-1}, 0, d_1, d_2\}$:

$$D(\tau) = \frac{1}{2}\Big[2d_0 + (-d_{-1} + d_1)\tau + (2d_{-1} - 5d_0 + 4d_1 - d_2)\tau^2 + (-d_{-1} + 3d_0 - 3d_1 + d_2)\tau^3\Big] \tag{3}$$

D. Backward Warping e Ricampionamento Bicubico

Per evitare fori o sovrapposizioni (*splatting artifacts*) generati dal forward warping, si adotta il campionamento all'indietro (*backward warping*). Per ogni coordinata intera (x, y) di I_τ, le coordinate sorgente non intere in I_0 e I_1 risultano:

$$(x'_0, y'_0) = (x, y) - D(\tau), \quad (x'_1, y'_1) = (x, y) - D(\tau) + d_1 \tag{4}$$

Il valore cromatico in (x', y') è interpolato spazialmente considerando l'intorno 4×4 (16 pixel) circostante $\lfloor x' \rfloor, \lfloor y' \rfloor$:

$$S(x', y') = \sum_{m=-1}^{2} \sum_{n=-1}^{2} I(\lfloor x' \rfloor + n, \lfloor y' \rfloor + m) \cdot C(x' - \lfloor x' \rfloor; n) \cdot C(y' - \lfloor y' \rfloor; m) \quad (5)$$

dove $C(t; k)$ rappresenta la funzione peso univariata di Catmull-Rom. Il valore sintetizzato finale scaturisce dal blending:

$$I_\tau(x, y) = (1 - \tau) \cdot S_0(x'_0, y'_0) + \tau \cdot S_1(x'_1, y'_1) \quad (6)$$

II. Implementazione e Modelli di Parallelizzazione

A. Architettura Sequenziale di Riferimento

La baseline CPU sequenziale è sviluppata in C++ standard ed esegue l'elaborazione single-thread di riferimento senza parallelizzazione. Il Block Matching sequenziale presenta una complessità asintotica per frame pari a:

$$\mathcal{O}\left(\frac{H \cdot W}{B^2} \times (2W + 1)^2 \times B^2\right) = \mathcal{O}(H \cdot W \cdot W^2) \quad (7)$$

Per un'immagine 4K con $W = 64$, la valutazione comporta circa 1.39×10^{11} iterazioni di confronto pixel.

B. Paradigmi e Ottimizzazioni CUDA

La parallelizzazione su GPU sfrutta la gerarchia di calcolo dell'architettura NVIDIA Pascal. Sono state sviluppate quattro varianti per la stima del moto e tre per l'interpolazione spaziale.

1) Block Matching: Varianti di Kernel:

- **GlobalOnly**: Ciascun blocco CUDA è mappato su un macro-blocco dell'immagine. I thread scandiscono lo spazio dei candidati (dx, dy) accedendo direttamente alla memoria globale sia per il frame di riferimento F_1 sia per il frame target F_2. L'elevata ridondanza di lettura genera saturazione del throughput della cache L2.
- **SharedF1**: Il macro-blocco di F_1 ($B \times B$ pixel) viene caricato cooperativamente in Shared Memory all'inizio del blocco. I thread leggono F_1 a latenza quasi nulla ($\approx$ 1–2 cicli di clock), azzerando il traffico di lettura ripetuta per il frame di ancoraggio.
- **SharedF1F2**: Entrambi i macro-blocchi vengono pre-caricati in memoria condivisa. Per F_2, viene trasferita l'intera regione di ricerca allargata $(B+2W) \times (B+2W)$ pixel. Tutti i confronti SAD avvengono internamente alla Shared Memory, massimizzando il riuso del dato.
- **WarpPerCandidate**: Parallelizzazione a grana fine in cui un intero warp (32 thread) valuta cooperativamente il SAD di un singolo vettore candidato, accumulando le differenze parziali tramite istruzioni di shuffle intra-warp (`__shfl_down_sync`), eliminando la contesa di memoria condivisa.

```
extern __shared__ Pixel s_mem[];
Pixel* s_F1 = s_mem;
Pixel* s_F2 = s_mem + block_w * block_h;

// Caricamento cooperativo di F1 (B x B)
for (int i = threadIdx.x; i < block_w * block_h; i += blockDim.x) {
    int r = i / block_w, c = i % block_w;
    s_F1[i] = d_F1[(y0 + r) * W + (x0 + c)];
}

// Caricamento cooperativo della finestra F2
int tile2_w = block_w + 2 * W;
int tile2_h = block_h + 2 * W;
for (int i = threadIdx.x; i < tile2_w * tile2_h; i += blockDim.x) {
    int r = i / tile2_w, c = i % tile2_w;
    int gy = max(0, min(H - 1, y0 - W + r));
    int gx = max(0, min(W - 1, x0 - W + c));
    s_F2[i] = d_F2[gy * W + gx];
}
__syncthreads();
```

Listing 1. Caricamento cooperativo in SharedF1F2

2) Vincolo Fisico della Shared Memory: La variante `SharedF1F2` richiede una quantità di memoria condivisa per blocco pari a:

$$M_{\text{shared}} = \left[B^2 + (B + 2W)^2\right] \times \text{sizeof(Pixel)} \quad \text{[Byte]} \quad (8)$$

Su architettura GP106, il limite hardware per blocco è di 48 KB (49152 byte). Con $\text{sizeof(Pixel)} = 4$ byte (RGBA), la condizione di eseguibilità risulta:

$$B^2 + (B + 2W)^2 \leq 12288 \quad (9)$$

Per $B = 16$ e $W = 64$, la memoria richiesta è $[256 + (16 + 128)^2] \times 4 = 83968$ byte ($\approx$ 82 KB), che eccede la capacità fisica dell'SM. Di conseguenza, il runtime CUDA rifiuta il lancio del kernel; tale vincolo architetturale giustifica l'assenza della variante nei benchmark per ampie finestre di ricerca.

3) Interpolazione Bicubica: Varianti di Kernel: Nella fase di sintesi del frame intermedio, tutti i pixel appartenenti allo stesso macro-blocco $B \times B$ condividono i medesimi vettori di moto e, di conseguenza, lo stesso spostamento temporale $D(\tau)$ e gli stessi 16 pesi spaziali bicubici. Le tre varianti implementate esplorano diversi compromessi tra ridondanza di calcolo, traffico verso la memoria globale e uso della Shared Memory:

- **Baseline**: Adotta una mappatura diretta in cui ogni thread calcola un singolo pixel di output (x, y). Ogni thread ricava autonomamente lo spostamento $D(\tau)$, gli indici interi di campionamento e i 16 pesi cubici di Catmull-Rom, per poi leggere direttamente dalla *Global Memory* i 16 pixel dell'intorno 4×4 in F_1 e i 16 in F_2 (applicando ad ogni lettura il controllo di bordo `clamp`). È l'approccio più semplice, ma ripete gli stessi calcoli polinomiali per tutti i B^2 pixel del macro-blocco ed effettua letture non coalescenti ripetute in DRAM.
- **Precomputed**: Elimina la ridondanza aritmetica suddividendo l'esecuzione in due kernel distinti. Nel primo passo viene lanciato un solo thread per ciascun macro-blocco di moto: esso calcola una volta sola gli offset interi e i pesi spaziali cubici, salvandoli in memoria globale in una struttura allineata (`BlockParams` da 80 byte). Nel secondo passo, ogni thread-pixel legge i parametri già pronti ed esegue direttamente il campionamento 4×4 in Global Memory e il blending finale.

- **SharedTile**: Modifica la granularità di esecuzione associando un intero blocco CUDA a ciascun macro-blocco $B \times B$ dell'immagine. Poiché tutti i pixel del macro-blocco subiscono lo stesso spostamento intero, i rispettivi intorni 4×4 si sovrappongono ampiamente, ricadendo in un'unica regione contigua di dimensione $(B+3) \times (B+3)$ pixel (un alone di 1 pixel a sinistra/alto e 2 pixel a destra/basso). Il kernel opera in tre fasi sincronizzate:
  1) un singolo thread (`threadIdx.x == 0`) calcola i pesi e gli offset del macro-blocco in *Shared Memory*;
  2) tutti i thread del blocco collaborano per caricare in *Shared Memory* i due tile $(B+3) \times (B+3)$ estratti da F_1 e F_2, eseguendo il `clamp` ai bordi una sola volta in questa fase;
  3) i thread elaborano i B^2 pixel di output leggendo i campioni 4×4 direttamente dalla *Shared Memory* ad altissima banda e bassa latenza.

  Anche questa variante è soggetta al limite fisico di 48 KB di Shared Memory per blocco ($2 \times (B+3)^2 \times 4 + 80 \leq 49152$ byte), risultando eseguibile fino a $B \leq 75$ e venendo esclusa per $B = 96$.

III. Metodologia Sperimentale di Benchmark

A. Setup Hardware

I test prestazionali sono stati condotti sul sistema descritto in Tabella I.

Tabella I
SPECIFICHE DELL'AMBIENTE DI ESECUZIONE

Componente	Caratteristica Tecnica
Sistema Host	ASUS ROG Strix GL503VM
CPU	Intel Core i7-7700HQ (4C/8T, max 3.80 GHz)
Memoria Host	16 GB DDR4-2400 MHz
GPU	NVIDIA GeForce GTX 1060 Mobile (GP106)
CUDA Cores	1280 Core (10 SM, Compute Capability 6.1)
VRAM	6 GB GDDR5 (192-bit, $\approx$ 192 GB/s)
Frame Input	4K UHD (3840×2160, 32-bit RGBA)

B. Protocollo di Misurazione

La profilazione temporale è integrata nel benchmark orchestrato dall'applicazione. Il campionamento temporale è effettuato su host tramite il clock ad alta precisione di `std::chrono`, racchiudendo l'intero ciclo di elaborazione della funzione di interfaccia. Poiché la sincronizzazione finale effettua il trasferimento sincrono dispositivo-host dei risultati tramite `cudaMemcpy`, la metrica registrata quantifica con rigore il tempo totale **End-to-End**:

$$T_{\text{total}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{overhead}} \tag{10}$$

Questo approccio fornisce una valutazione realistica del sistema in ambiente di produzione, tenendo conto dei colli di bottiglia del bus PCIe Gen3 x16 ($\approx$ 12 GB/s pratici).

IV. Analisi Critica dei Risultati

A. Interpolazione Lineare (LERP)

La Tabella II sintetizza le prestazioni dell'algoritmo LERP al variare del numero di frame intermedi generati ($n \in \{1, 4, 8\}$).

Tabella II
PRESTAZIONI ALGORITMO LERP (RISOLUZIONE 4K UHD)

n	BlkX	T_{seq} (ms)	T_{cuda} (ms)	σ_{cuda}	Speedup	MaxDiff
1	64	119.87	39.01	1.59	3.07×	0
1	256	119.87	38.28	0.35	3.13×	0
1	1024	119.87	38.48	0.67	3.11×	0
4	64	467.78	123.10	0.44	3.80×	0
4	256	467.78	123.31	0.55	3.79×	0
4	1024	467.78	123.44	0.63	3.79×	0
8	64	943.93	231.91	1.85	4.07×	0
8	256	943.93	231.76	1.61	4.07×	0
8	1024	943.93	232.08	1.38	4.07×	0

Valutazione: Lo speedup del LERP converge asintoticamente a circa $4\times$. Tale comportamento è governato dalla legge di Amdahl applicata a kernel memory-bound: ciascun frame 4K RGBA pesa $3840 \times 2160 \times 4 \approx 33.18$ MB. L'invocazione con $n = 1$ implica il trasferimento di 2 frame in ingresso (66.36 MB) e 1 in uscita (33.18 MB) per un totale di circa 100 MB su bus PCIe. A circa 12 GB/s, il tempo minimo di trasferimento teorico è di ≈ 8.3 ms per la sola memoria, a cui si sommano allocazioni e latenze di interrupt. La GPU elabora il frame in meno di 2 ms, rendendo il canale PCIe il collo di bottiglia predominante.

B. Block Matching Standalone

Il confronto tra le quattro varianti di Block Matching per macro-blocchi $B \in \{16, 32, 64, 96\}$ al variare di W è riportato in Tabella III.

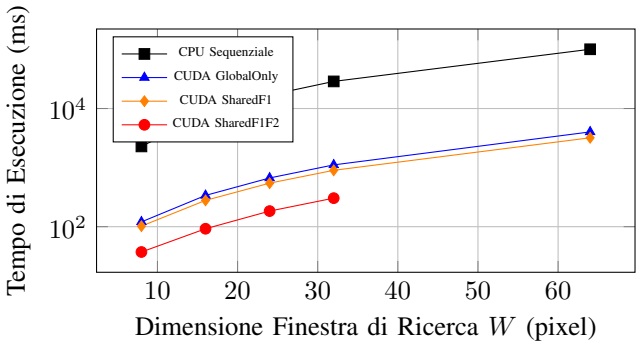


Figura 1. Tempi di calcolo in scala logaritmica per il Block Matching ($B = 16$) al crescere del raggio della finestra di ricerca W.

Analisi delle prestazioni:

1) **Efficacia del Tiling in Shared Memory**: La variante `SharedF1F2` distanzia nettamente tutte le altre implementazioni, mantenendo un tempo di calcolo di appena 304 ms su frame 4K con $W = 32$, contro i circa 28.7 secondi della CPU, registrando uno speedup di $\mathbf{94.56}\times$.

Tabella III
CONFRONTO VARIANTI BLOCK MATCHING AL VARIARE DI B E W

B	W	Variante	BlkX	T_{seq} (ms)	T_{cuda} (ms)	Speedup
16	8	GlobalOnly	256	2283.4	120.58	18.94×
16	8	SharedF1	256	2283.4	102.19	22.34×
16	8	SharedF1F2	256	2283.4	37.28	**61.24**×
16	8	WarpPerCand	256	2283.4	96.64	23.63×
16	16	GlobalOnly	256	7878.5	338.26	23.29×
16	16	SharedF1	256	7878.5	278.72	28.27×
16	16	SharedF1F2	256	7878.5	92.27	**85.39**×
16	16	WarpPerCand	256	7878.5	317.31	24.83×
16	24	GlobalOnly	256	16877.3	667.63	25.28×
16	24	SharedF1	256	16877.3	545.55	30.94×
16	24	SharedF1F2	256	16877.3	183.72	**91.87**×
16	24	WarpPerCand	256	16877.3	681.90	24.75×
16	32	GlobalOnly	256	28766.7	1111.3	25.89×
16	32	SharedF1	256	28766.7	899.76	31.97×
16	32	SharedF1F2	256	28766.7	304.21	**94.56**×
16	32	WarpPerCand	256	28766.7	1174.7	24.49×
16	64	GlobalOnly	256	100659	4018.6	25.05×
16	64	SharedF1	256	100659	3195.8	31.50×
16	64	SharedF1F2	–	100659	*N/A (smem)*	*N/A*
16	64	WarpPerCand	256	100659	4519.9	22.27×
32	8	SharedF1F2	256	1842.5	36.26	**50.81**×
32	16	SharedF1F2	256	6397.2	87.96	**72.73**×
32	32	SharedF1F2	256	22877.1	299.44	**76.40**×
64	8	SharedF1F2	256	1717.5	41.57	**41.32**×
64	16	SharedF1	256	5983.5	257.01	23.28×
64	64	GlobalOnly	256	72158.8	3822.0	18.88×
96	8	GlobalOnly	256	1662.8	109.67	15.16×
96	8	SharedF1	256	1662.8	158.90	10.46×
96	32	GlobalOnly	256	21408.3	1017.6	21.04×
96	64	GlobalOnly	256	76483.9	3756.0	20.36×

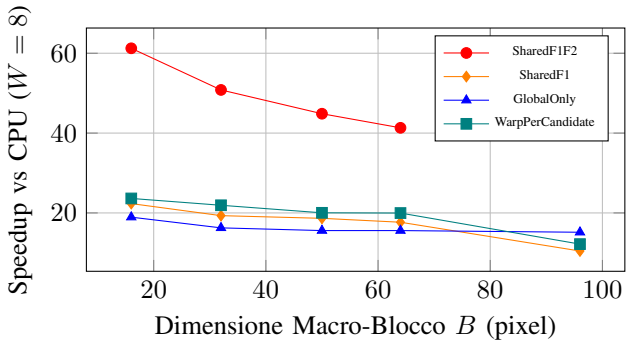


Figura 2. Andamento dello speedup per $W = 8$ al variare di B. Blocchi grandi riducono il numero di blocchi nella griglia, degradando il parallelismo.

La memorizzazione locale del frame target evita cache thrashing su L2.

2) **Inversione di efficienza in WarpPerCandidate**: Sebbene teoricamente elegante, `WarpPerCandidate` mostra tempi leggermente superiori a `GlobalOnly` per blocchi ridotti e W elevati (4519 ms vs 4018 ms a $W = 64$). La suddivisione delle coordinate tra thread dello stesso warp introduce divergenza di diramazione nei controlli di bordo (`clamp`) e overhead di sincronizzazione shuffle non ammortizzato.
3) **Impatto della dimensione** B: All'aumentare di B da 16 a 96 (Figura 2), il numero di macro-blocchi totali scende da 32400 a 920. La riduzione della quantità di blocchi attivi nella griglia riduce l'occupancy complessiva degli SM GPU; per $B = 96$, l'ingombro di Shared Memory per blocco impedisce l'esecuzione di `SharedF1F2` e rallenta `SharedF1`.
4) **Accuratezza e conformità numerica**: Le metriche di divergenza tra CPU e GPU (`max_diff` e `px_diff_gt1`) risultano identicamente nulle (0) in tutte le configurazioni testate, certificando la completa equivalenza matematica dell'elaborazione parallela rispetto al flusso sequenziale.

C. Pipeline Bicubica Completa e Scaling Temporale ($n \in \{1, 4, 8\}$)

L'indagine quantitativa della pipeline bicubica completa (stima del moto sui 3 target + warping e ricampionamento spaziotemporale) al variare del numero di frame intermedi generati è sintetizzata in Tabella IV.

Tabella IV
SCALING TEMPORALE DELLA PIPELINE BICUBICA ($B = 16, W = 8$, BLK 32×4)

n	BM Variant	Bicubic Variant	T_{seq} (ms)	T_{cuda} (ms)	Speedup
1	GlobalOnly	Baseline	9640.99	329.88	29.23×
1	SharedF1	Baseline	9640.99	281.57	34.24×
1	SharedF1F2	Baseline	9640.99	112.90	85.39×
1	SharedF1F2	SharedTile	9640.99	107.88	**89.37**×
1	WarpPerCand	Baseline	9640.99	286.61	33.64×
4	GlobalOnly	Baseline	19419.10	392.14	49.52×
4	SharedF1	Baseline	19419.10	346.18	56.09×
4	SharedF1F2	Baseline	19419.10	178.28	108.92×
4	SharedF1F2	SharedTile	19419.10	172.43	**112.62**×
4	WarpPerCand	Baseline	19419.10	351.32	55.27×
8	GlobalOnly	Baseline	31685.50	481.25	65.84×
8	SharedF1	Baseline	31685.50	433.31	73.12×
8	SharedF1F2	Baseline	31685.50	264.97	119.58×
8	SharedF1F2	SharedTile	31685.50	259.45	**122.13**×
8	WarpPerCand	Baseline	31685.50	438.68	72.23×

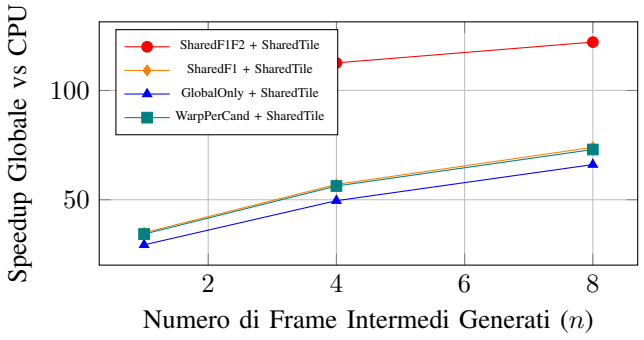


Figura 3. Crescita dello speedup al variare di n. L'ammortamento della stima del moto esalta il throughput dell'interpolazione GPU.

Analisi del fenomeno di ammortamento: Dai dati sperimentali emerge una proprietà fondamentale: lo speedup cresce in modo monotono all'aumentare di n (Figura 3). La spiegazione risiede nella struttura a due fasi della pipeline:

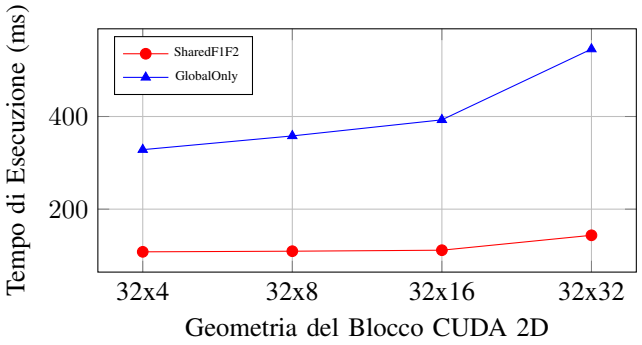


Figura 4. Sensibilità del tempo di esecuzione bicubico ($n = 1$) alla geometria dei blocchi 2D. A 1024 thread (32×32) si verifica saturazione dei registri.

1) **Fase di Stima del Moto (fissa)**: I 3 vettori di spostamento vengono calcolati una sola volta all'avvio. Con `SharedF1F2`, questa fase impegna circa 86 ms su GPU e circa 6380 ms su CPU.
2) **Fase di Interpolazione (scalabile con n)**: Per ciascun frame intermedio, la GPU esegue il kernel bicubico su 3840×2160 thread in circa 21.6 ms (inclusivo di trasferimento PCIe D2H), mentre la CPU impiega oltre 3100 ms per frame.

Lo speedup marginale della sola fase di interpolazione risulta dunque pari a $\frac{3100}{21.6} \approx 143.5\times$. Di conseguenza, al crescere di n, il peso relativo della fase di interpolazione diventa predominante rispetto alla stima del moto iniziale, spingendo lo speedup globale da $89.37\times$ ($n = 1$) fino a $\mathbf{122.13}\times$ ($n = 8$).

Divergenza numerica e considerazioni di precisione: Nei benchmark della pipeline bicubica si riscontra un valore di `max_diff` pari ad 1, con `px_diff_gt1 = 0`. Tale discrepanza infinitesima (± 1 LSB su canali a 8 bit $[0, 255]$) è fisiologica nei calcoli grafici paralleli: scaturisce dalla diversa modalità di arrotondamento della FPU x86 a 64-bit rispetto all'unità FMAD (Fused Multiply-Add) dei CUDA Core a 32-bit nelle operazioni di combinazione lineare:

$$x \cdot y + z \quad \text{vs} \quad \text{fmaf}(x, y, z) \tag{11}$$

L'operazione FMAD esegue moltiplicazione e accumulo con un singolo troncamento finale, garantendo una precisione superiore ma non bit-exact rispetto all'istruzione sequenziale CPU scorrelata.

D. Sensibilità alla Geometria dei Blocchi CUDA

L'esplorazione bidimensionale della griglia CUDA (32×4, 32×8, 16×16, 32×32), mostrata in Figura 4, rivela che configurazioni con 128 o 256 thread per blocco massimizzano l'occupancy degli SM Pascal:

- I blocchi 32×4 (128 thread) e 32×8 (256 thread) offrono il compromesso ideale tra allocazione di registri per thread e partizionamento della memoria condivisa.
- Blocchi con 1024 thread (32×32) causano una marcata degradazione delle prestazioni (oltre il 33% di rallentamento, ad es. `SharedF1F2` passa da 107.8 ms a 143.5 ms), a causa della saturazione dei banchi di registri per SM e del conseguente calo del numero di warp attivi schedulabili simultaneamente (*warp occupancy limitation*).

V. Conclusioni e Sviluppi Futuri

L'indagine quantitativa dimostra l'elevata scalabilità del framework CUDA nell'elaborazione di flussi video ad altissima risoluzione. La transizione da un approccio naive in Global Memory ad un modello di memoria a gerarchia esplicita con tiling in Shared Memory (`SharedF1F2`) costituisce il fattore cardine per convertire algoritmi memory-bound in carichi ad elevato throughput, consentendo un'accelerazione fino a due ordini di grandezza ($> 122\times$) rispetto ad architetture general-purpose.

Sviluppi futuri prevedono l'adozione di texture memory bidimensionali hardware per sfruttare il filtro di interpolazione bilineare integrato nelle Texture Mapping Units (TMU) della GPU, nonché l'integrazione di stream asincroni CUDA multipli per sovrapporre il trasferimento PCIe del frame $k + 1$ con la computazione del frame k.